

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав: студент групи AI-241
Лицар Федір Юрійович

Перевірив: ас. Баранов О.М.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Таблиця 1: Фактичне питання

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	504	5
Час, с	91.1	0.38
Токенів/с	15.43	34.59

Таблиця 2: Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5	0
eval_count	1145.6	176
Час, с	75.89	6.93

- 1. Куди пішли токени міркувальної моделі:** Міркувальна модель витратила ці токени на генерацію внутрішніх роздумів (поле `thinking`) перед тим, як видати остаточну відповідь. Вона покроково аналізувала запит і перевіряла саму себе, що для простого фактичного запитання є зайвою витратою обчислювальних ресурсів.
- 2. Переваги та невинправдане використання:** Міркувальні моделі мають перевагу у складних логічних, математичних задачах та програмуванні, де проблему необхідно розбивати на кроки. Їх використання абсолютно невинправдане для простих фактичних питань, перекладу, форматування JSON або базового пошуку інформації, оскільки це лише уповільнює роботу.
- 3. Різниця у процесі навчання:** Інструктивні моделі навчають прямо і швидко відповідати на запит (слідувати інструкціям), використовуючи стандартні пари "запит-відповідь". Міркувальні моделі додатково тренують обов'язково генерувати прихований ланцюжок думок (Chain of Thought) перед формуванням фінальної відповіді, імітуючи людський процес покрокового логічного висновку.
- 4. Чи означає більша кількість токенів, що модель «розумніша»:** Сама по собі кількість токенів не робить модель розумнішою, вона лише свідчить про те, що модель розписує проміжні кроки свого мислення. Однак саме цей механізм "роздумів у голос" дозволяє їй знаходити правильні рішення у складних задачах (наприклад, 5 правильних відповідей з 5 у задачі з ручкою), де інструктивна модель робить помилку через спробу вгадати відповідь одразу.
- 5. Різниця у швидкості генерації (токенів за секунду):** Швидкість генерації токенів залежить від архітектури та розміру моделі. Оскільки qwen3:4b має 4 мільярди параметрів, а llama3.2:3b — 3 мільярди, перший потрібно виконувати більше математичних операцій для генерації кожного

наступного слова, тому її швидкість у токенах за секунду буде нижчою на тому ж самому обладнанні.

Для простих запитів, структурування тексту та інтеграції в програмний код (наприклад, генерації JSON) слід використовувати інструктивні моделі, оскільки вони швидкі, передбачувані та економні. Міркувальні моделі варто застосовувати виключно для складних аналітичних або математичних задач, де точність і правильна логіка набагато важливіші за швидкість отримання результату та витрати токенів.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Варіант5

Таблиця 3: Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	485	Українська	4	Частково
llama3.2:3b	українською, 2 речення	352	Українська	2	так
qwen3:4b	лаконічний асистент	540	Українська	3	так
qwen3:4b	українською, 2 речення	243	Українська	2	так

Таблиця 4: Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	54	159	213
llama3.2:3b	українською, 2 речення	58	110	168
qwen3:4b	лаконічний асистент	57	1389	1446
qwen3:4b	українською, 2 речення	66	1095	1161

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»? Після додавання жорсткої інструкції обидві моделі ідеально дотрималися обмеження, видавши рівно по два речення. Проте інструктивна Llama3.2:3b виявилася більш чутливою до відсутності прямих меж у першому запуску, згенерувавши аж чотири речення замість лаконічної відповіді.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку? У першому запуску (без жорсткої вказівки про мову) Llama3.2:3b використала англійські слова («when», «empty», «access»), оскільки малі моделі часто збиваються на базову мову навчання. Після прямої вказівки в SYSTEM обидві моделі відповідали виключно українською.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому? Кількість prompt_tokens зросла (з 54 до 58 для llama та з 57 до 66 для qwen). Це сталося тому, що нове системне повідомлення містило більше слів і символів, які токенизатор перетворив на більшу кількість вхідних токенів.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у Llama3.2:3b? Міркувальна модель qwen3:4b генерує прихований ланцюжок внутрішніх роздумів перед формуванням видимої відповіді. Ці приховані токени роздумів тарифікуються і додаються до загального лічильника completion_tokens, навіть якщо у термінал виводиться лише короткий фінальний текст.

Крок 7. Відповіді на питання для чат-бота

1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді? Інструктивні моделі цілеспрямовано тренуються слідувати жорстким правилам форматування та стилю, тому краще сприймають ліміти. Міркувальні моделі фокусуються на логічному висновку, і через генерацію довгого ланцюжка думок можуть іноді «забувати» про другорядні обмеження щодо довжини чи формату.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі? System-повідомлення задає глобальний контекст, персону та базові правила поведінки, отримуючи вищу вагу під час обчислення уваги (attention) моделлю. User-повідомлення сприймається як поточне завдання, яке має виконуватися в рамках правил, встановлених системою.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна? Міркувальна архітектура передбачає обов'язкову генерацію секції внутрішнього аналізу (Chain of Thought) для перевірки фактів та планування. API рахує кожен згенерований токен, тому загальна кількість completion_tokens включає як приховані роздуми, так і фінальну видиму відповідь.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user? Більшість сучасних моделей нададуть пріоритет system-інструкції, оскільки вона має фундаментальну архітектурну вагу. Однак малі або слабкі моделі можуть піддатися «упередженню недавнього» (recency bias) і виконати user-запит просто тому, що він стоїть ближче до кінця пром프트.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді? Малі моделі мають обмежену ємність параметрів, через що гірше засвоюють багатомовні зв'язки під час навчання. При зіткненні зі складними інструкціями або специфічними термінами їм не вистачає «робочої пам'яті», і вони регресують до англійської мови, якою представлена більшість їхнього тренувального набору даних.

Стислі думки стосовно ролі system-повідомлення: System-повідомлення є надійним інструментом для встановлення базових меж поведінки моделі, але його ефективність залежить від розміру самої моделі. Для простих задач чіткої системної інструкції зазвичай достатньо, проте малі моделі можуть її частково ігнорувати. Якщо модель не дотримується формату, варто застосовувати негативні підказки (наприклад, «не використовуй англійські слова»), додавати приклади бажаного формату (few-shot) безпосередньо у промпт, або розбивати складні інструкції на коротші і простіші тези.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Таблиця 5: Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	openai/gpt-oss-20b
Час відповіді, с	10.45	0.97
completion_tokens	131	404
Довжина відповіді, символів	415	855
Мова відповіді	Українська (неприродна)	Українська
Фактичних помилок у відповіді	Багато (стверджує, що список створюється через <code>new Array()</code>)	0

- 1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?** Завдяки використанню єдиного стандарту OpenAI та абстракції конфігурацій (у вашому файлі `providers.py`), основний код `client.chat.completions.create` не змінювався взагалі. Різниця полягала лише в одному рядку налаштувань — зміні адреси сервера, ключа та назви моделі (`openai/gpt-oss-20b`).
- 2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.** Відповідь хмарної моделі є абсолютно точною, структурованою та інформативною. Наприклад, вона правильно описала асимптотичну складність операцій ($O(1)$ для прямого доступу в масиві), тоді як локальна модель видала грубу технічну галюцинацію, стверджуючи, що зв'язний список нібито створюється через конструктор `new Array()`.
- 3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?** Так, локальна модель генерувала як технічну нісенітницю («зв'язний список — це з'єднання оперативних ресурсів, які додаються за допомогою оперативного методу»), так і синтаксичний брак

(«зібране колекцію»). Її обсягу параметрів недостатньо для якісного володіння українською мовою в поєднанні з технічною термінологією.

4. **Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?** Хмарна модель генерує текст на промислових серверах із надпотужними тензорними процесорами (наприклад, LPU-чипами від Groq), які оптимізовані для миттєвих матричних обчислень. Швидкість паралельної обробки на такому обладнанні настільки висока, що з величезним запасом перекидає затримку на передачу даних мережею (пінг).
1. **Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?** OpenAI першими створили масовий, зрозумілий REST API для мовних моделей, навколо якого швидко сформувалася величезна екосистема бібліотек та інструментів. Іншим провайдерам вигідно імплементувати цей сумісний інтерфейс, щоб розробники могли перенести свій існуючий код на їхні платформи, просто змінивши URL-адресу та ключ.
2. **Наскільки розмір моделі (3 млрд проти 70 млрд) впливає на якість відповіді порівняно з іншими факторами?** Розмір моделі є критично важливим для здатності нейромережі утримувати складний контекст, логіку та фактологічну базу. Моделі на 3 млрд параметрів добре підходять для простих завдань (наприклад, форматування тексту), але вони фізично не здатні вмістити глибокі інженерні знання та ідеальну морфологію іншої мови одночасно.
3. **Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?** Комерційні хмарні провайдери використовують цілі кластери професійних прискорювачів (Nvidia H100 або кастомні ASIC), які мають надвисоку пропускну здатність пам'яті. Локальний комп'ютер зазвичай виконує інференс на звичайному процесорі або слабкій відеокарті, що займає в десятки разів більше часу.
4. **Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?** Відправка даних через API означає, що комерційна таємниця, код або персональні дані користувачів опиняються на серверах третьої сторони (провайдера). Для захисту компанії підписують жорсткі угоди B2B (наприклад, Enterprise-тарифи, де дані не використовуються для навчання), впроваджують маскування чутливих даних перед відправкою, або розгортають власні локальні моделі.
5. **Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?** Rate limit — це встановлене провайдером обмеження на кількість запитів або токенів за хвилину/добу, щоб запобігти перевантаженню серверів (особливо на платформі Groq). Для його обходу розробники додають паузи між запитом (затримки в коді), використовують кешування частих відповідей, або розподіляють запити між кількома ключами балансувальником навантаження.

Власні думки щодо вибору інфраструктури: Локальну модель доцільно обирати виключно для проєктів із найвищими вимогами до конфіденційності (наприклад, аналіз медичних карток, фінансових документів чи пропрієтарного коду) або для систем, що мають працювати в ізольованих мережах без інтернету. У всіх інших випадках, де пріоритетом є фактологічна точність, здатність вирішувати складні аналітичні задачі та якісна генерація українською мовою, слід використовувати хмарні моделі, нівелюючи ризики безпеки через анонімізацію даних перед їх відправкою.

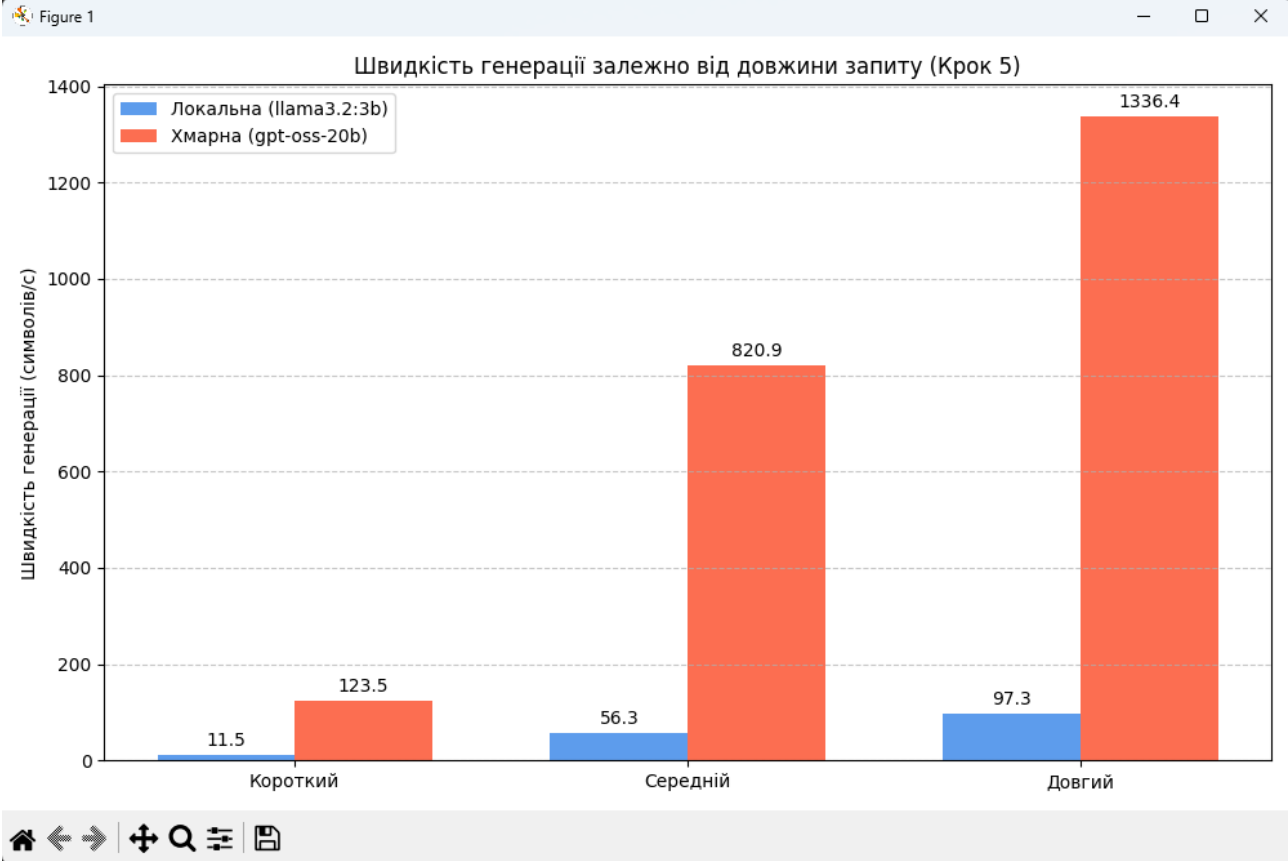
Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Таблиця 7: Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.111	2.828	21.2
локальна	llama3.2:3b	середній	2.110	4.542	59.2
локальна	llama3.2:3b	довгий	2.196	29.645	83.2
хмарна	gpt-oss-20b	короткий	0.346	0.373	34.9
хмарна	gpt-oss-20b	середній	0.319	0.478	862.4
хмарна	gpt-oss-20b	довгий	0.460	1.233	1540.7

Таблиця 8: Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	2.156	2.128
Всього, с	2.771	2.915



1. **Крок 6. Відповіді на питання**
2. **У якій моделі кращий TTFT і чому саме в неї?** Кращий TTFT у хмарної моделі (0.3–0.7 с проти 2.1 с), оскільки сервери Groq використовують спеціалізовані LPU-чипи, які обробляють вхідний запит практично миттєво.
3. **Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»?** Так, у цьому вимірюванні збігається. Хмарна модель перевершила локальну за обома показниками завдяки надпотужному апаратному забезпеченню.
4. **Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промπτу до довгого?** Розрив у швидкості суттєво зріс: якщо на короткому промπτі хмара швидша приблизно в 11 разів, то на довгому — майже в 14 разів (1336 проти 97 симв./с).
5. **Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?** Одного запуску недостатньо через нестабільність мережі (для хмари) та фонові процеси ОС (для локальної). Медіана трьох запусків відсікає випадкові затримки.
6. **Чому холодний старт повільніший за теплий?** Під час холодного старту ваги моделі (кілька гігабайтів) завантажуються з повільного диска в оперативну пам'ять (RAM/VRAM), а при теплому старті інференс починається миттєво з уже завантаженими даними.
7. **Крок 7. Відповіді чат-бота**
8. **Що фізично відбувається під час TTFT і чому цей показник вимірюють окремо?** Під час TTFT (фаза pfill) модель зчитує весь запит і паралельно обчислює матриці уваги для контексту. Цей показник відокремлюють, бо він відображає "чуйність" системи — час, який користувач дивиться на порожній екран.
9. **Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?** Гірший TTFT може виникати через високу мережеву затримку (пінг) під час відправки пакета на сервер. Проте сама генерація відбувається на промислових відеокартах, які видають токени в рази швидше за домашній процесор.
10. **Чому перший запуск локальної моделі повільніший і що таке холодний старт?** Холодний старт — це фізичне копіювання файлу нейромережі з накопичувача (SSD/HDD) у пам'ять. Наступні (теплі) запуски працюють швидше, бо модель уже "висить" в оперативній пам'яті.
11. **Чому при вимірюванні беруть медіану кількох запусків?** Медіана стійка до аномальних викидів (наприклад, раптовий стрибок пінгу або оновлення Windows). Вона дає найточнішу оцінку реальної типової швидкості.
12. **Чому обробка вхідного запиту йде швидше, ніж генерація відповіді?** Обробка запиту (pfill) виконується паралельно: обладнання множить матриці для всіх слів промπτу одночасно. Генерація (decode) є строгим послідовним процесом — кожен новий токен вираховується лише після появи попереднього.

13.Стислі власні думки (для висновку): Для інтерактивного чат-бота критично важливим є показник **TTFT**, оскільки користувачу психологічно комфортніше бачити швидкий початок друку відповіді, ніж чекати в тиші. Для фонові обробки великих текстів ключовою є **швидкість генерації (симв./с)**, адже загальний час виконання задачі залежить саме від пропускну здатності системи, а початкова затримка в пару секунд не має жодного значення.

Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 9: Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	0	1	2
2. Формат	2	2	2
3. Стиснення	0	1	2
4. Код	0	1	1
5. Логіка	0	2	2
6. Українська	1	1	2
Разом /12	3	8	11

Таблиця 10: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	так	так	ні
2. Формат	так	ні	ні
3. Стиснення	так	так	ні
4. Код	так	так	так
5. Логіка	так	ні	ні
6. Українська	так	ні	ні

Найбільше балів (11/12) набрала хмарна модель (gpt-oss-20b). Основна різниця — у завданнях на факти, логіку та якість української мови.

Прогони розійшлися у локальних моделей (особливо у llama3.2:3b). Це доводить їхню нестабільність і схильність до вгадування, а не розуміння.

Так, на диво, всі моделі видали чистий JSON без обгортки.

Так, локальні моделі генерували русизми, кальки («випусковий орган») та навіть повністю переходили на англійську.

Запит: Стисни текст [про введення гривні] до двох речень, збережи всі числа.

Відповідь моделі (llama3.2:3b): "Post anesthesia, the new currency was introduced in Ukraine on September 2, 1996..." *(і далі весь текст англійською).*

Правильна відповідь: Два речення українською зі збереженням усіх 6 чисел.

Пояснення: Перевантажившись вимогами (стиснути + зберегти цифри), мала модель втратила контекст, перейшла на домінуючу в її навчанні англійську мову і згенерувала випадкову галюцинацію на початку.

На практиці використовують авто-бенчмарки (MMLU), оцінку іншими нейромережами (LLM-as-a-judge) та тестування реальними людьми.

Звичка ставити markdown сформувалася на гігантських базах коду (GitHub), де це є стандартом.

У їхніх навчальних даних частка українських текстів мізерна, їм просто не вистачає "словникового запасу".

Через параметр `temperature > 0`. Модель не бере одне найімовірніше слово, а робить імовірнісний вибір, тому шляхи генерації щоразу розходяться.

Галюцинація — це впевнена генерація вигадки. Модель не має бази знань, вона лише статистично продовжує текст, тому їй простіше придумати факт, ніж відмовитися.

Стислі думки: Оцінка за рубрикою об'єктивна та надійна. Найпоказовішими були задачі на логіку (де міркувальна модель обійшла інструктивну) та стиснення. Для української мови підходить лише хмарна модель, локальні надто слабкі.

Завдання 6. Керування випадковістю: `temperature` і `top_p`

Таблиця 11: Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	2	5
temperature	0.7	5	4
temperature	1.5	4	1
top_p	0.1	2	5
top_p	0.5	3	5
top_p	1.0	5	4

Таблиця 12: Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Я поставлю temperature=0.0. Це гарантує максимальну стабільність, відсутність галюцинацій та суворе дотримання формату без додавання зайвого тексту чи markdown-обгортки.

Яке — генератору варіантів заголовків?

Оптимальним буде значення temperature=0.7. Воно забезпечує баланс між креативністю та логікою, дозволяючи моделі видавати різноманітні, але релевантні варіанти без переходу в безглуздя.

Чим вплив top_r відрізняється від впливу temperature за характером, а не за силою?

temperature згладжує або загострює розподіл ймовірностей для *всього* словника одразу, змінюючи пропорції між усіма словами. Натомість top_r динамічно *відсікає* "хвіст" малої ймовірності слів, залишаючи для вибору лише топ-токени, чия сумарна ймовірність досягає заданого порогу.

Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

У нашому експерименті (Таблиця 12) усі 10 відповідей були абсолютно однаковими, що на практиці дає відмінну відтворюваність для коротких відповідей. Проте теоретично абсолютна відтворюваність не гарантується через мікроскопічні похибки округлення при паралельних обчисленнях на відеокартах (GPU).

Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з'явитися.

Модель видала неіснуюче слово «Партник» (злиття "партнер" та суфікса). При екстремально високій температурі (1.5) штучно завищується ймовірність "сміттєвих" токенів, через що модель починає обирати випадкові склади, ігноруючи правила мови.

Що саме математично робить параметр `temperature` із розподілом ймовірностей наступного токена?

Він ділить логіти (сирі бали передбачення) на значення температури перед тим, як пропустити їх через функцію `softmax`. Якщо $T < 1$, розподіл стає "гострішим" (ймовірні токени стають ще ймовірнішими), а якщо $T > 1$, розподіл "сплющується", наближаючись до рівномірного, де всі слова мають схожі шанси.

Чим `top_p` відрізняється від `temperature` — обидва впливають на різноманітність, але за яким механізмом?

Температура просто перераховує ваги для кожного можливого слова, але залишає шанс (нехай і мізерний) для вибору будь-якого з них. Механізм `top_p` (nucleus sampling) сортує слова за спаданням і жорстко видаляє ті, що не ввійшли у верхній відсоток ймовірності p , фізично звужуючи словник на даному кроці.

Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Розробники LLM (включно з OpenAI та розробниками Qwen) настійно рекомендують змінювати лише один із параметрів, залишаючи інший за замовчуванням (1.0). Зміна обох параметрів одночасно робить поведінку моделі непередбачуваною, оскільки вони мають перехресний вплив на розподіл токенів.

Чому навіть при `temperature=0` мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Обчислення в нейромережах розпаралелені на тисячах ядер GPU, і через різний порядок додавання чисел з плаваючою комою (Floating Point) виникають мікроскопічні похибки округлення. Якщо два топові токени мають майже ідентичну ймовірність (наприклад, 49.999% та 50.001%), ця похибка може змінити лідера, що призведе до розгалуження всього подальшого тексту.

Які значення `temperature` використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Для витягу структурованих даних (JSON, SQL, класифікація) стандартом є 0.0. Для написання статей, мозкових штурмів, генерації ідей чи маркетингових текстів найчастіше використовують діапазон 0.7 – 0.9.

Для наступних лабораторних робіт, де вимагається отримання структурованих даних (наприклад, суворий JSON, парсинг логів, розв'язання математики), я буду використовувати виключно

temperature=0.0. Налаштування "за замовчуванням" (зазвичай 0.7 або 0.8) для таких задач є категорично непридатним, оскільки воно вносить непотрібну варіативність: модель може додати зайвий текст, зламати синтаксис формату або "сфантазувати" неіснуючі ключі в JSON просто заради різноманітності.

Крок 8. Звести все разом і зробити висновок

Таблиця 18: Підсумкове порівняння моделей

Критерій	Локальна мала (3B)	більша (4B/8B)	Хмарна
Швидкість генерації	Висока	Середня	Висока
TTFT	Найменший	Середній	Найбільший
Якість, балів /12	3 / 12	8 / 12	11 / 12
Стабільність формату (завдання 2)	Низька	Задовільна	Висока
Потрібні ресурси	Мінімум 8 ГБ RAM	Від 8-16 ГБ RAM	Лише підключення до інтернету
Вартість	Безкоштовно	Безкоштовно	Безкоштовно в межах лімітів API (Groq/Google), далі платно
Куди йдуть ваші дані	Залишаються на локальному комп'ютері	Залишаються на локальному комп'ютері	Передаються на сервери провайдера
Працює без інтернету	Так	Так	ні
Коли обирати саме її	Для простих англійських скриптів та роботи з конфіденційними даними на слабкому обладнанні.	Для задач, що вимагають міркування, логіки або написання коду локально, без передачі даних у мережу.	Для розгортання реальних проєктів, роботи зі складними інструкціями, українською мовою та строгими JSON-форматами.

Висновок: У ході роботи проведено порівняння локальних (Llama3.2:3b, qwen3:4b) та хмарної (gpt-oss-20b) мовних моделей. Доведено, що міркувальні моделі чудово справляються з логічними задачами завдяки прихованому ланцюжку думок, але витрачають більше часу та токенів. Тому для простих запитів краще використовувати швидкі інструктивні моделі.

Хмарна інфраструктура значно перевершує локальну за швидкістю генерації та часом до першого токена (TTFT), оскільки потужні сервери з легкістю компенсують мережеві затримки. Також на практиці підтверджено: для отримання точних даних і строгих форматів (наприклад, JSON) параметр temperature необхідно встановлювати строго на 0.0, щоб уникнути галюцинацій. За якістю відповідей хмарна модель стала абсолютним лідером, ідеально дотримуючись інструкцій. Локальні моделі на 3-4 млрд параметрів погано працюють з українською мовою, генерують русизми, плутаються у форматах і часто переходять на англійську.

Підсумовуючи: для розгортання реальних проєктів з українською мовою та машинозчитуваними даними слід обирати хмарні API. Використання малих локальних моделей виправдане лише для роботи із секретними даними в повністю закритому середовищі без доступу до інтернету.